Lecture Notes on Dictionary in Python

Dictionary :

A dictionary is a mutable, ordered (Python 3.7+) collection of key–value pairs. While keys must be unique and immutable (like strings or integers), values can be of any data type, whether mutable or immutable. This makes a dictionary ideal for accessing data using keys instead of positions.

Syntax:

```
Dict = {"Key" : "Value"}
```

Example:

```
Height = {"Ram" : 160, "Mohan" : 165, "Rao" :  172}
```

Properties:

- Dictionaries are dynamic
- Dictionary Keys are hashable objects like int, string, tuple
- Dictionaries are mapping types implemented using hash tables and therefore support key-based access rather than positional indexing

Application:

- Used to create structured data such as profiles
- Used for grouping data
- Used for data mapping
- Used for passing keyword arguments to functions in Python
- Used for counting occurrences and many other applications

The concept of Hashing:

- Hashing is a technique used to convert a value into a fixed integer called a hash value, which is used for data access.
- A hash function takes an input and returns an integer (Hash value)
- Hash value of an object does not change during program execution
- Hashable objects are typically immutable and have a fixed hash value
- Python uses a hash table to implement dictionaries and sets
- A Python dictionary does not search sequentially; instead it retrieves values using hash values
- Immutability is necessary for hashability, but not sufficient

When a dictionary is created using unique keys, the keys are assigned hash values where the associated values are stored in the memory. When called using the keys, Python retrieves it straight from the locations.

If, D = {"A " : 1} internally, "A " → some number (hash value)

That number is the index of the memory where 1 is stored.

How to Create a Dictionary:

A dictionary can be created in several ways.

Example 1:

D = {} (It creates an empty dictionary)

Example 2:

D = {"A" : 1, "B" : 2, "C" : 3, "D": 4}

Example 3:

D = dict(A=1, B=2, C=3, D=4)

Example 4:

D = dict([("A", 1), ("B", 2), ("C", 3), ("D", 4)])

Example 5:

Given, Alphabets = ["A", "B", "C", "D"], Positions = [1, 2, 3, 4]

D = dict(zip(Alphabets, Positions))

Manipulating a Dictionary:

The following gives some of the ways to manipulate the dictionaries.

Let D = {"A": 1, "B": 2, "C": 3, "D": 4}

(a) Replace the value associated with a key:

To access the value associated with the key "B", use D["B"].

```
>> print(D["B"])    # 2.
```

If you want to change the value associated with the key "B" as 5, for instance, the statement D["B"] = 5 will change the value.

```
>> D["B"] = 5
>> print(D)          # D = {"A" : 1, "B" : 5, "C" : 3, "D" : 4}
```

(b) Adding new items to a dictionary:

```
>> D["E"] = 5
>> print(D)          # D = {"A" : 1, "B" : 2, "C" : 3, "D" : 4, "E" : 5}
```

(c) Deleting an existing item in a dictionary:

```
>> del D["C"]
```

```
>> print(D)          # D = {"A" : 1, "B" : 2, "D": 4}
```

(d) Deleting a dictionary:

```
>> del D
>> print(D)          # NameError: name "D" is not defined
```

Common Dictionary methods:

1. Getting Data(Accessing values):

When we want to **read data** from a dictionary, we can use the following built-in methods.

- get(key) → get a value associated with the key

```
>> print(D.get("A"))          # 1
>> print(D.get("A", 22))      # 1
>> print(D.get("G"))          # None
>> print(D.get("G", 111))     # 111
```

- keys() → get all keys

```
>> print(D.keys())            # dict_keys(['A', 'B', 'C', 'D'])
```

- values() → get all values

```
>> print(D.values())          # dict_values([1, 2, 3, 4])
```

- items() → get key-value pairs

```
>> print(D.items())           # dict_items([('A', 1), ('B', 2), ('C', 3), ('D', 4)])
```

2. Adding & Updating Data:

We can use the following when we **add new items or change existing ones**.

- update() → Add or update multiple values

```
>> D.update({'C': 33})
>> print(D)                          # { "A" : 1, "B" : 2, "C" : 33, "D": 4}
>> D.update({'B': 222, 'E': 5})      # { "A" : 1, "B" : 222, "C" : 33, "D": 4, "E": 5 }
```

- setdefault() → Add only if key doesn't exist else returns the value

```
>> print( D.setdefault('A', 2))      # 1
>> print(D)                          # { "A" : 1, "B" : 2, "C" : 3, "D": 4}
>> print( D.setdefault('E', 5))      # 5
>> print(D)                          # { "A" : 1, "B" : 2, "C" : 3, "D": 4,
                                        "E":5}
```

3. Removing Data:

We can use the below mentioned methods to delete items from the dictionary.

- pop(key) → Removes specific item

```
>> print(D.pop('B'))                 # 2
>> print(D.pop('E'))                 # KeyError because "E" does not exist
>> print(D.pop('C', None))           # 3
>> print(D.pop('E', None))           # None
```

- popitem() → Removes last inserted item

```
>> print(D.popitem())          #("D", 4) [key, value pair]
>> print(D)                    # {"A" : 1, "B" : 2, "C" : 3}
```

- clear() → Removes the entire dictionary items

```
>> D.clear()
>> print(D)                    # {} [ An empty dictionary]
```

4. Copying Dictionary:

To **duplicate a dictionary,** we will use the copy() method.

- copy() → creates a shallow copy

- shallow copy creates a new object, but the elements inside it are references to the original objects. Therefore, changes to mutable elements inside the copy will affect the original, but changes to the outer structure will not.

```
>> d = {"a" : [1, 2, 3]}
>> copy_d = d.copy()                [creates shallow copy]
>> copy_d["a"].append(4)            [trying to append 4 to the list value]
>> print(copy_d)          # {"a" : [1, 2, 3, 4]}
>> print(d)               # {"a" : [1, 2, 3, 4]}
```

In this case, attempt to modify the value in the shallow copy modified the original dictionary “d” itself because list is mutable object and has reference to the original.

```
>> d = {"a" : [1, 2, 3]}
>> copy_d = d.copy()                [creates shallow copy]
```

```
>> copy_d["a"] = [111, 444]          [trying to assign a new list]

>> print(copy_d)                     # {"a" : [111, 444]}

>> print(d)                          # {"a" : [1, 2, 3]}
```

Now, the original dictionary “d” is not modified because, we have created a new reference for the key “a” of the shallow copy.

5. Creating Dictionary:

The following is a special method to create a new dictionary. It assigns the same value to all the keys.

- fromkeys(keys, value) → create dictionary with same values

fromkeys() has two arguments, where keys can be any iterable like list, tuple, string, set.

```
>> D = dict.fromkeys(["a", "b", "c"], 5)

>> print(D)                          # {"a" : 5, "b" : 5, "c": 5}
```

6. Dictionary Comprehension:

Dictionary comprehension is used to create a dictionary in a short and clear way. It allows keys and values to be generated from a loop in one line. This helps in building dictionaries directly without writing multiple statements.

```
>> D = { i : i for i in [1, 2, 3, 4]}   #  {1 : 1, 2 : 2, 3 : 3, 4 : 4}

>> D = { f "{i}" : i**2 for i in range(1, 4)}

                                        # {"1" : 1, "2" : 4, "3" : 9}
```

Let s = [10, 20, 30, 40, 50]

```
>> D = { f"{i//10}" : i for i in range(100) if i in s}
```

```
#  {"1" : 10, "2" : 20, "3" : 30, "4" : 40, "5" : 50}
```

Key points to remember or practice:

- Some dictionary methods support default values (get, setdefault)
- Most dictionary methods return view or values (e.g. keys(), values(), items()), while some modify the dictionary in place.
- get() and setdefault() functions may look similar but they are distinct
- Use of del and pop() are not the same
- Understanding a shallow copy is important

Looping through a Dictionary:

Iterations over dictionaries can be performed over keys, values or key-value pairs using iterable views. Let us create a new dictionary for understanding the looping technique with examples.

In a class of 6 students coming from different cities, we shall create a dictionary with their names as ‘keys’ and their home towns as ‘values’.

```
Home_town = {"Raghav" : "Mumbai",  "Dev" : "Nagpur",
             "Madhan" : "Pune", "Sachin" : "Mumbai",
            "Johan" : "Delhi", "Vijay": "Pune"}
```

1. Looping through all the keys in a dictionary:

We can loop through the keys of a dictionary to perform operations using dictionary keys. For example, If we want to print only ‘keys’ of a dictionary, we will use this method.

```
>> for name in Home_town.keys():
        print(name.title())
```

The above piece of code will print all the keys in the dictionary ‘Home_town’ as shown below.

```
Raghav
Dev
Madhan
Sachin
Johan
Vijay
```

Important note *: Looping through the keys is actually the default behavior when looping through a dictionary.*

```
>> for name in Home_town:

        print(name.title())
```

The code above will also yield the same result.

```
Raghav
Dev
Madhan
Sachin
Johan
Vijay
```

2. Looping through all the Values in a Dictionary:

As in the similar way of looping through all ‘keys’ of a dictionary, we can even loop through all values of a dictionary as explained below.

```
>> for city in Home_town.values():

        print(city.title())
```

The above piece of code will print all the values in the dictionary ‘Home_town’ as shown below.

```
Mumbai
Nagpur
Pune
```

Mumbai
Delhi
Pune

If we want the list of cities from which the students are participating, then we have to remove duplicates.

```
>> print(list(dict.fromkeys(list(Home_town.values()))))
```

Mumbai
Nagpur
Pune
Delhi

3. Looping through items in a Dictionary:

```
>> for k, v in Home_town.items():        [k, v for key and value]
>>      print(k, v)
```

The output is,

Raghav	Mumbai
Dev	Nagpur
Madhan	Pune
Sachin	Mumbai
Johan	Delhi
Vijay	Pune

Nesting concept in Dictionary:

Nesting, in general, refers to placing one data structure inside another.

1. A list of Dictionaries:

As its name suggests, in this type, a list is formed using a number of dictionaries. Let us say, there are five dictionaries named dict1, dict2, dict3, dict4 and dict5 and one list referred to as ‘L’.

L = [dict1, dict2, dict3, dict4, dict5]

We shall see a detailed example by considering 3 students details.

```
Student_1 = {"name" : "Ranjith", "gender" : "Male", "class" : "Class – V"}

Student_2 = {"name" : "Shalini", "gender" : "Female", "class" : "Class – IV"}

Student_3 = {"name" : "Sharma", "gender" : "Male", "class" : "Class – VI"}
```

My_list = [Student_1, Student_2, Student_3]

My_list will contain a list of three dictionaries which are pertaining to three different students.

2. A list inside a Dictionary:

When we need to have a list of values corresponding to each key, value can have a list.

```
Dict = {"key": ["value1", "value2", "value3"]}
```

Let us consider, we need to deal with the students having different subjects.

```
Students = {"Ramu" : ["English", "Hindi", "Commerce"],
            "Somu" : ["English", "French", "History"],
            "Rinku" : ["Marathi", "Science", "Geography"]
            }
```

In the above example, students may have different subjects. When accessed using a key, the value will be a list instead of a string.

3. A Dictionary in a Dictionary:

In this type of dictionary, we will have a dictionary in place of values corresponding to each key. While creating a profile, this type of nesting can be used to effectively to store information about staff, student etc.

```
Dict ={"KEY1" : {"key1": "value1", "key2": "value2"}}
```

Let us create a profile of staff members in an organization using a dictionary as explained above.

```
Profile = {
        "111" : {"Name" : "Tirupati",
                 "Designation" : "Professor",
                 "Gender" : "Male"
                 },
        "222" : {"Name" : "Anitha",
                  "Designation" : "Reader",
                 "Gender" : "Female"
                }
}
```

After creating the above dictionary, when you access the key ‘111’, it will give you a dictionary instead of a single key value.

```
>> print(Profile["111"])
    {
     "Name" : "Tirupati",
     "Designation" : "Professor",
      "Gender" : "Male"
    }
```

If we want to get the name of the employee whose profile number is ‘111’, then we have to access the key [‘name’] after we have pulled the dictionary associated with the profile number ‘111’.

```
>> print(Profile["111"]["Name"])
```

OR

```
>> print(Profile["111"].get("Name"))
```

Will give us,

Tirupati

List vs Tuple vs Dictionary (Python Comparison Table)

Feature	List	Tuple	Dictionary
Syntax	[ ]	( )	{key: value}
Example	[1, 2, 3]	(1, 2, 3)	{'a': 1, 'b': 2}
Type	Sequence	Sequence	Mapping
Mutability	Mutable (can change)	Immutable (cannot change)	Mutable
Order	Ordered	Ordered	Ordered (Python 3.7+)
Access	By index	By index	By key
Indexing	Yes	Yes	No
Duplicates	Allowed	Allowed	Keys must be unique
Memory Usage	More	Less	More
Speed	Moderate	Faster	Very fast lookup
Methods	Many methods	Limited methods	Many methods
Iteration	Easy	Easy	Easy
Nested Support	Yes	Yes	Yes
Use as Dict Key	No	Yes (if immutable)	Not applicable
Best Use Case	Dynamic data	Fixed data	Key-value data

Quick Summary:

- **List** → Flexible and changeable
- **Tuple** → Fixed and faster
- **Dictionary** → Fast key-based access